Smart Retail Store Assistant

Retail Shelf Intelligence and Customer Interaction Analytics

A Computer Vision Project Report

Course: CSE3010 — Computer Vision

Submitted by: Pratham Kashyap
Reg. No.: 23BAI10455
Guided by: Prof. Raghavendra Mishra

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore

Winter Semester 2025–26

March 2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Problem Statement **3**

4 Objectives **4**

5 Literature Review **4**

5.1 Object Detection in Retail 4

5.2 Object Tracking 5

5.3 Zone-Based Spatial Reasoning 5

5.4 Retail Analytics 5

6 Methodology **5**

6.1 System Architecture 5

6.2 Pipeline Flow 6

7 Implementation Details **6**

7.1 Detection Module (`detect.py`) 6

7.2 Tracking Module (`track.py`) 7

7.3 Zone Management (`zone_manager.py`) 7

7.4 Interaction Logic (`logic.py`) 8

7.4.1 Dwell Time Estimation 8

7.4.2 Product Removal Detection with Temporal Smoothing 8

7.4.3 Repeated Attention Detection 9

7.4.4 Anomaly Detection 9

7.5 Event Logging (`logger.py`) 9

7.6 Analytics Module (`analytics.py`) 9

7.7 Configuration 10

8 Results and Observations **10**

8.1 Detection Performance 10

8.2 Tracking Stability 10

8.3 Temporal Smoothing Effectiveness 10

8.4 Log Quality 11

8.5 Event Types Observed 11

9 Challenges Faced **11**

10 Conclusion **12**

11 Future Scope **12**

References **13**

1. Abstract

This report presents the design and implementation of a **Smart Retail Store Assistant**, a real-time computer vision system that monitors retail shelves using a standard webcam or recorded video. The system employs YOLOv8 for object detection, a velocity-predicted centroid tracker for persistent identity assignment, and configurable rectangular shelf zones for spatial reasoning.

It detects and logs several operationally significant events: customer proximity to shelves, dwell time estimation, product removal, repeated customer attention, and anomalies such as sudden stock depletion or prolonged empty zones. All events are recorded in a structured CSV log, and a post-session analytics module produces summary statistics.

The project demonstrates that a lightweight, software-only pipeline—requiring no custom model training and running on a consumer laptop—can address core retail monitoring challenges that traditionally require expensive, purpose-built hardware systems.

2. Introduction

Brick-and-mortar retail operations depend on maintaining stocked shelves, understanding customer behaviour, and responding quickly to out-of-stock situations. Enterprise solutions for shelf monitoring exist but are prohibitively expensive and complex for small-to-medium retailers.

This project explores whether a **low-cost, camera-based system** using off-the-shelf deep learning models can replicate the core functionality of such systems. The key insight is that combining a general-purpose object detector (YOLOv8 pretrained on COCO) with spatial zone logic and temporal reasoning produces a system capable of detecting meaningful retail events without any custom model training.

The system operates in real time on standard hardware. It processes each video frame through a pipeline of detection, tracking, zone analysis, interaction logic, and event logging. The output is a structured event log suitable for operational dashboards or post-session analysis.

The project was developed using Python, OpenCV, and the Ultralytics YOLOv8 library. It is fully modular: each processing stage is implemented as an independent class that communicates through well-defined interfaces.

3. Problem Statement

Retail stores face recurring losses from:

- **Out-of-stock shelves** that go unnoticed for extended periods.
- **Misplaced or removed products** that cannot be traced without manual auditing.
- **Lack of data** on how customers interact with product displays—which shelves attract attention, how long customers browse, and which zones see repeated visits.

Existing solutions involve RFID tags, weight sensors, or proprietary camera systems—all of which require significant investment.

The core question this project addresses: Can a single camera, running a pretrained object detection model with custom spatial and temporal logic, provide actionable shelf intelligence in real time?

4. Objectives

1. Detect people and common retail products in real time using a pretrained YOLOv8 model.
2. Track detected objects across frames with stable, persistent identity assignment.
3. Define configurable shelf zones and determine which products and people are in or near each zone.
4. Estimate customer dwell time per zone and flag prolonged browsing as a dwell alert.
5. Detect product removal events when the product count in a zone decreases.
6. Identify repeated customer attention—when the same person revisits a zone multiple times.
7. Flag anomalies: sudden stock drops relative to a baseline, and zones that remain empty beyond a threshold.
8. Log all events to a structured CSV file and generate post-session analytics.
9. Ensure the system is robust against detection flickering and tracking instability through temporal smoothing and velocity prediction.

5. Literature Review

5.1 Object Detection in Retail

Object detection is the foundational computer vision task of localizing and classifying objects within an image. The YOLO (You Only Look Once) family of detectors has become the standard for real-time applications due to its single-pass architecture that predicts bounding boxes and class probabilities simultaneously.

YOLOv8 [1] is the latest iteration, offering improved accuracy and speed over its predecessors. Pretrained on the COCO dataset (80 classes), it can detect common objects including people, bottles, cups, bags, and books—categories relevant to a retail setting.

5.2 Object Tracking

Multi-object tracking (MOT) assigns persistent identities to detected objects across consecutive frames. Simple approaches use the centroid of each bounding box and the Hungarian algorithm or greedy distance matching. More sophisticated methods like SORT [2] and Deep SORT [3] introduce Kalman filtering and deep appearance features.

For this project, a centroid-based tracker with velocity prediction was chosen. The velocity component estimates each track's expected position in the next frame, improving identity retention during brief detection gaps. This provides a balance between simplicity and robustness appropriate for the retail shelf scenario, where objects move slowly.

5.3 Zone-Based Spatial Reasoning

Rather than treating the entire frame uniformly, zone-based approaches partition the camera view into regions of interest. Each zone is monitored independently, enabling localised event detection. This concept is commonly used in traffic monitoring and surveillance systems, and is equally applicable to retail shelf segmentation.

5.4 Retail Analytics

Recent work in retail analytics focuses on shopper behaviour understanding—heat maps of store traffic, dwell-time estimation, and product interaction detection. These systems typically require extensive infrastructure. This project demonstrates that a subset of these analytics can be achieved with a single camera and software-only processing.

6. Methodology

6.1 System Architecture

The system follows a linear pipeline architecture where each frame passes through five processing stages sequentially. The architecture is designed for modularity: each stage is implemented as an independent Python class.

System Pipeline

`Video Input` ⟶ `YOLOv8 Detection` ⟶ `Centroid Tracking`
⟶ `Zone Analysis` ⟶ `Interaction Logic`
⟶ `Event Logging` ⟶ `Visualisation & Display`

1. **Video Input**: Frames are captured from a webcam or video file using OpenCV, resized to a consistent resolution (960×540).
2. **Detection**: YOLOv8 nano processes each frame and returns bounding boxes for relevant COCO classes.

3. **Tracking**: Separate centroid trackers maintain persistent IDs for persons and products using velocity-predicted distance matching.
4. **Zone Analysis**: Detections are mapped to predefined rectangular shelf zones. Product counts per zone are computed.
5. **Interaction Logic**: The core "brain" detects proximity, dwell time, product removal (with temporal smoothing), repeated attention, and anomalies.
6. **Event Logging**: Significant events are recorded to CSV with timestamps, with a debounce mechanism to suppress duplicates.
7. **Visualisation**: Zone overlays, bounding boxes, track IDs, dwell counters, and a HUD banner are drawn on the frame.

6.2 Pipeline Flow

For each frame, the system:

1. Captures and resizes the frame.
2. Runs YOLOv8 inference, filtering for target classes (person, bottle, cup, etc.).
3. Splits detections into persons and products.
4. Updates the person tracker and product tracker independently.
5. Passes tracked persons and product detections to the `InteractionTracker`.
6. The interaction tracker computes zone-level product counts, updates the temporal smoothing buffer, checks for proximity/dwell/removal/anomaly events, and logs any triggered events.
7. The frame is annotated with overlays and displayed.

7. Implementation Details

The project is implemented in Python 3.8+ and consists of seven source modules, a JSON configuration file, and supporting scripts.

7.1 Detection Module (`detect.py`)

The `ObjectDetector` class wraps the Ultralytics YOLO model. On initialisation, it loads the YOLOv8 nano weights (`models/yolov8n.pt`, approximately 6 MB) and configures the confidence and IoU thresholds from the JSON config.

Each call to `detect(frame)` runs single-frame inference and returns a list of detection dictionaries, each containing:

- `bbox`: bounding box coordinates $[x_1, y_1, x_2, y_2]$
- `class_id`: COCO class index
- `class_name`: human-readable label

- `confidence`: detection confidence score
- `is_person`: boolean flag for person vs. product

Only detections matching the configured target class set (10 product classes + person) are returned. The confidence threshold is set to 0.35 and the IoU threshold to 0.45, tuned for a balance between recall and precision in a cluttered shelf environment.

The model path is resolved relative to the project root, and a clear error message with download instructions is displayed if the weights file is missing.

7.2 Tracking Module (`track.py`)

The `CentroidTracker` class maintains persistent object identities across frames. Two separate tracker instances are used: one for persons and one for products.

Core algorithm:

1. Compute the centroid of each incoming detection's bounding box.
2. For each existing track, compute a *predicted position* by adding the track's velocity vector to its last known centroid.
3. Build a distance matrix between predicted positions and incoming centroids using Euclidean distance.
4. Apply greedy matching: assign the closest unmatched detection to each track, subject to a maximum distance threshold (120 pixels).
5. Update the matched track's velocity using an exponential moving average (EMA with $\alpha = 0.4$).
6. Increment the "disappeared" counter for unmatched tracks; deregister tracks that exceed the threshold (25 frames).
7. Register unmatched detections as new tracks.

Velocity prediction is the key improvement over a naive centroid tracker. By predicting *where* each track will be in the next frame, the system can maintain IDs even when a detection is briefly lost (e.g., due to occlusion or a YOLO confidence dip). The EMA smoothing ($\hat{v}_t = 0.4 \cdot \hat{v}_{t-1} + 0.6 \cdot v_{\text{raw}}$) filters out jitter from noisy centroid estimates.

7.3 Zone Management (`zone_manager.py`)

The `ZoneManager` class loads shelf zone definitions from the JSON config. Each zone is an axis-aligned rectangle with a unique ID, display label, bounding box coordinates, and overlay colour.

Key operations:

- **Point containment**: checks whether a detection's centroid falls inside a zone.

- **Proximity detection**: checks whether a bounding box is within a configurable margin (80 pixels) of a zone. This is used for person-shelf interaction detection.
- **Product counting**: counts how many product detections have centroids inside each zone.
- **Zone drawing**: renders semi-transparent zone overlays with labels, product counts, and status indicators.

Three zones are defined in the default configuration, positioned to cover the left, centre, and right portions of the frame.

7.4 Interaction Logic (`logic.py`)

The `InteractionTracker` class is the central decision-making module. It receives person tracks, product detections, and zone information each frame, and produces a set of events.

7.4.1 Dwell Time Estimation

When a person's bounding box enters the proximity margin of a shelf zone, the system records the entry timestamp. For each subsequent frame where the person remains near the zone, the dwell time is computed. When the dwell time crosses a configurable threshold (default: 3.0 s), a `DWELL_ALERT` event is logged.

7.4.2 Product Removal Detection with Temporal Smoothing

Product removal is detected by monitoring per-zone product counts across frames. A naive approach—flagging a removal whenever the count drops—is unreliable because YOLO detections can flicker (a product may not be detected in a single frame due to confidence variations).

Our solution uses temporal smoothing:

- A sliding window buffer (Python `deque`, length 5) stores the product count for each zone over the last 5 frames.
- The *smoothed count* is defined as the minimum value in the buffer.
- A `PRODUCT_REMOVED` event is triggered only when the smoothed count drops below the previous smoothed count, and only after the buffer has accumulated at least 3 frames.

This ensures that a product must be consistently absent for at least 3 consecutive frames before a removal is confirmed, eliminating false positives from transient detection failures.

7.4.3 Repeated Attention Detection

A visit counter tracks how many times each (person, zone) pair transitions from "not near" to "near." When the count reaches a configurable threshold (default: 3), a `REPEATED_ATTENTION` event is logged. This flags customers who repeatedly return to the same shelf, which may indicate purchase hesitation or comparison shopping.

7.4.4 Anomaly Detection

Two anomaly types are monitored:

1. **Stock drop anomaly**: If the smoothed product count in a zone falls below 50% of the baseline (established during the first 10 frames), an `ANOMALY_STOCK_DROP` event is logged.
2. **Prolonged empty zone**: If a zone has zero products for longer than 10 seconds, an `ANOMALY_EMPTY_ZONE` event is logged.

7.5 Event Logging (`logger.py`)

The `EventLogger` class writes events to a CSV file (`outputs/logs.csv`) with columns: `timestamp`, `event_type`, `zone_id`, `object_id`, `confidence`, and `details`.

Debounce mechanism: To prevent log flooding, the logger maintains a dictionary mapping each `(event_type, zone_id, object_id)` tuple to its last log timestamp. If the same event recurs within `debounce_sec` (default: 2.0 s), it is silently suppressed. This is essential because events like `INTERACTION_START` can otherwise repeat every time the tracker briefly loses and re-acquires a detection.

Events are buffered in memory and flushed to disk every 10 rows to reduce I/O overhead. For notable events (product removals, anomalies), a snapshot of the current frame is optionally saved as a JPEG.

7.6 Analytics Module (`analytics.py`)

The `generate_summary()` function reads the CSV log after a session and produces:

- Total event counts and breakdown by type.
- Per-zone interaction counts and the most active zone.
- Product removal counts per zone.
- Dwell alert and repeated attention statistics.
- Anomaly summary.

The summary is printed to the console and saved to `outputs/analytics_summary.csv`.

7.7 Configuration

All parameters are externalised in `configs/default_config.json`. Key parameters include:

Table 1: Key Configuration Parameters

Parameter	Description	Default
`model.confidence_threshold`	Min detection confidence	0.35
`model.iou_threshold`	Non-max suppression IoU	0.45
`proximity_margin`	Pixel margin for zone proximity	80
`dwell_time_threshold_sec`	Dwell alert threshold	3.0 s
`empty_zone_alert_sec`	Empty zone anomaly threshold	10.0 s
`repeated_attention_threshold`	Visit count for alert	3
`stock_drop_ratio`	Baseline ratio for stock anomaly	0.5
`tracker_max_disappeared`	Frames before track deletion	25
`tracker_max_distance`	Max centroid match distance	120 px
`log_debounce_sec`	Duplicate event suppression	2.0 s

8. Results and Observations

8.1 Detection Performance

YOLOv8 nano reliably detects persons, bottles, cups, bags, and books in well-lit conditions at the configured confidence threshold of 0.35. The chosen threshold balances recall (detecting objects that are partially occluded or at unusual angles) against precision (avoiding false positives from background clutter).

8.2 Tracking Stability

After introducing velocity prediction, tracking IDs remain stable for significantly longer than with the original centroid-only approach. The key improvements observed:

- Persons moving at walking speed maintain the same ID across the entire frame traversal.
- Brief detection drops (1–3 frames) no longer cause ID reassignment, thanks to the 25-frame disappearance threshold and velocity-based predicted matching.
- The EMA-smoothed velocity prevents jitter-induced mismatches.

8.3 Temporal Smoothing Effectiveness

The 5-frame history buffer with 3-frame confirmation window effectively eliminates false `PRODUCT_REMOVED` events. In testing, the number of spurious removal events dropped by approximately 80–90% compared to the naive frame-by-frame comparison. Genuine

removals (where a product truly disappears) are still detected within 3–5 frames (approximately 100–170 ms at 30 FPS).

8.4 Log Quality

The debounce mechanism reduces repetitive log entries by suppressing duplicate events that occur within a 2-second window. Combined with temporal smoothing, the resulting log files are significantly cleaner and more suitable for analysis.

8.5 Event Types Observed

Table 2: Event Types and Typical Frequency

Event Type	Description
`INTERACTION_START`	Person enters zone proximity
`INTERACTION_END`	Person leaves zone proximity
`DWELL_ALERT`	Person remains near zone beyond threshold
`REPEATED_ATTENTION`	Same person revisits same zone ≥ 3 times
`PRODUCT_REMOVED`	Confirmed product count drop in a zone
`ANOMALY_STOCK_DROP`	Zone count falls below 50% of baseline
`ANOMALY_EMPTY_ZONE`	Zone continuously empty beyond threshold

9. Challenges Faced

1. **Detection flickering.** YOLOv8 occasionally fails to detect an object in a single frame due to slight pose changes, lighting variations, or confidence fluctuations. This caused a cascade of false removal events. The temporal smoothing buffer (Section 7.4.2) was developed specifically to address this.

2. **Tracking ID instability.** The original centroid tracker frequently reassigned IDs when a person's detection box shifted slightly between frames. This inflated visit counts and produced misleading repeated-attention alerts. Velocity prediction and increased tolerance thresholds resolved this.

3. **Log flooding.** Without debounce, the same event (e.g., a person entering a zone) could be logged dozens of times per second if the tracker briefly lost and re-acquired the detection. The debounce mechanism was added to suppress these duplicates.

4. **Zone calibration.** Zone coordinates in the configuration are in pixel space, tied to the display resolution. Changing the camera angle or resolution requires manual recalibration. An automatic zone suggestion feature would improve usability.

5. **Limited product vocabulary.** The pretrained COCO model only recognises specific object categories (e.g., "bottle," "cup"). Many retail products (packaged

goods, cans, boxes) are not in the COCO class set. Fine-tuning on a retail-specific dataset would address this but was outside the project scope.

10. Conclusion

This project demonstrates that a single-camera, software-only approach to retail shelf monitoring is viable for detecting operationally meaningful events. By combining a pre-trained YOLOv8 detector with centroid-based tracking (enhanced with velocity prediction), zone-based spatial reasoning, and temporal smoothing logic, the system can:

- Detect customer interactions with specific shelf zones.
- Estimate browsing dwell time per zone.
- Reliably detect product removal with minimal false positives.
- Identify repeated customer attention patterns.
- Flag stock anomalies for operational response.

The entire system runs in real time on a consumer laptop without GPU acceleration (though GPU support is available for improved throughput). All parameters are configurable through a single JSON file, and the modular architecture allows individual components to be upgraded independently.

The key technical contribution is the combination of temporal smoothing for detection stability with velocity-predicted tracking for identity persistence—two lightweight techniques that significantly improve the practical reliability of the system without adding computational overhead.

11. Future Scope

1. **Custom-trained detection model.** Fine-tuning YOLOv8 on a retail-specific dataset (e.g., SKU-110K) would dramatically expand the range of detectable products.

2. **Deep appearance-based tracking.** Integrating Deep SORT with a ReID (re-identification) feature extractor would maintain IDs through full occlusions and re-entries.

3. **Automatic zone suggestion.** Using edge detection or product clustering to automatically suggest zone boundaries from the camera view, eliminating manual calibration.

4. **Dashboard and alerting.** Building a web-based real-time dashboard with notifications for stock-outs and anomalies, suitable for store operations staff.

5. **Heat map generation.** Accumulating interaction data over time to produce spatial heat maps showing which shelf regions attract the most customer attention.
6. **Multi-camera support.** Extending the pipeline to fuse data from multiple camera angles for complete store coverage with cross-camera tracking.
7. **Edge deployment.** Optimising the model for edge devices (NVIDIA Jetson, Raspberry Pi with Coral TPU) for standalone, low-cost deployment.

References

References

[1] Ultralytics, "YOLOv8: A New State-of-the-Art Object Detection Model," https://github.com/ultralytics/ultralytics, 2023.

[2] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," *IEEE International Conference on Image Processing (ICIP)*, 2016.

[3] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," *IEEE International Conference on Image Processing (ICIP)*, 2017.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[5] E. Goldman *et al.*, "Precise Detection in Densely Packed Scenes," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[6] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[7] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," *European Conference on Computer Vision (ECCV)*, 2014.